RESEARCH-ARTICLE

Porting Software Libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS

BO ZHOU, Northeastern University, Shenyang, Liaoning, China

JIAQI SHI, Northeastern University, Shenyang, Liaoning, China

YING WANG, Northeastern University, Shenyang, Liaoning, China

LI LI, Beihang University, Beijing, China

TSZ ON LI, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

HAI YU, Northeastern University, Shenyang, Liaoning, China

View all

Published: 22 June 2025
Accepted: 31 March 2025
Received: 22 February 2025

Proceedings of the ACM on Software Engineering, Volume 2, Issue ISSTA (June 2025)
https://doi.org/10.1145/3728941
EISSN: 2994-970X

Porting Software Libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS

BO ZHOU, JIAQI SHI, and YING WANG*, Northeastern University, China
LI LI, Beihang University, China
TSZ ON LI, The Hong Kong University of Science and Technology, China
HAI YU and ZHILIANG ZHU, Northeastern University, China

OpenHarmony emerges as a potent force in the mobile app domain, poised to stand alongside established industry giants. ArkTS is its main language, enhancing TypeScript (TS) and JavaScript (JS) with strict typing for improved performance. Developers are encouraged to port popular TS/JS libraries to OpenHarmony, supported by detailed guidelines. However, this requires a deep understanding of ArkTS syntax, following porting specifications, and making manual changes. An automated solution is crucial to streamline this process and foster a robust software ecosystem.

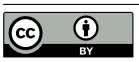
As a new programming language, ArkTS currently lacks essential analysis tools for automated analysis and porting of software libraries. However, the rise of Large Language Models (LLMs) shows promise for effectively addressing automated porting tasks. There are two challenges in using LLMs to automate the porting of TS/JS libraries to OpenHarmony: (1) *LLMs have limited exposure to ArkTS code, making it difficult for them to grasp the syntactical differences between ArkTS and JS/TS, as well as the various adaptation scenarios.* (2) *Project-level code adaptation often involves correcting numerous syntax mismatches, which complicates matters for LLMs as they must handle the interactions between different mismatches and interdependent code.* In response, we introduce `ArkAdapter`, a project-level automatic code adaptation approach. `ArkAdapter` addresses *Challenge 1* by establishing an adaptation knowledge repository for ArkTS syntax comprehension. It expands a collection of real code adaptation examples based on expert experience across various scenarios, improving the adaptation capabilities of LLMs through few-shot learning. `ArkAdapter` overcomes *Challenge 2* based on an adaptation priority strategy by considering both the dependency structure and the granularity of syntax-mismatching code. This strategy helps prevent interference among various syntax mismatches and their interdependent code. Evaluation shows `ArkAdapter` achieves high precision (86.84%) and full recall rates (100%) in locating syntax-mismatching code. 80.14% code adaptations inferred by `ArkAdapter` with LLM exactly or plausibly match the actual adaptations made by OpenHarmony developers. We utilized `ArkAdapter` to port 79 widely-used TS/JS libraries. **70 ArkTS libraries (88.6%) were validated by *ArkCompiler*, passed tests, and were approved by the reviewers of the OpenHarmony's library central repository**, showcasing its potential to streamline the porting process and enhance the growth of the OpenHarmony software landscape.

CCS Concepts: • **Software and its engineering → Software libraries and repositories**.

Additional Key Words and Phrases: LLM; Software Libraries; OpenHarmony; ArkTS; Software Adaptation

*Ying Wang is the corresponding author of this paper.

Authors' Contact Information: Bo Zhou, 2410606@stu.neu.edu.cn; Jiaqi Shi, 1745564873@qq.com; Ying Wang, wangying@swc.neu.edu.cn, Northeastern University, Shenyang, Liaoning, China; Li Li, lilicoding@ieee.org, Beihang University, Beijing, China; Tsz On Li, toli@connect.ust.hk, The Hong Kong University of Science and Technology, Hong Kong, China; Hai Yu, yuhai@mail.neu.edu.cn; Zhiliang Zhu, zhuzhiliang_neu@163.com, Northeastern University, Shenyang, China.



ACM 2994-970X/2025/7-ARTISSTA064
https://doi.org/10.1145/3728941

ACM Reference Format:
Bo Zhou, Jiaqi Shi, Ying Wang, Li Li, Tsz On Li, Hai Yu, and Zhiliang Zhu. 2025. Porting Software Libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA064 (July 2025), 22 pages. https://doi.org/10.1145/3728941

1 Introduction

> ***The OpenHarmony ecosystem sets sail with a thousand ships: with many followers, the journey is boundless.***

OpenHarmony is the foundation of Harmony OS, an open-source mobile platform launched by the *OpenAtom Foundation* [17] after receiving open-source code from Huawei [28]. With the release of *HarmonyOS NEXT* in October 2024, the platform has nearly one billion users, suggesting it could create a competitive balance with Android and iOS in the mobile app market. Although many popular applications have been ported to OpenHarmony, the platform's software supply chain is still in its early stages. As of October 1, 2024, OpenHarmony's Library Central Repository, *OHPM* [16], has released only 839 third-party libraries, while the *npm* central repository has over three million. In the long term, building a diverse and robust software library ecosystem is essential for empowering mobile app developers to thrive within the OpenHarmony community.

ArkTS is its main language, enhancing TypeScript (**TS**) and JavaScript (**JS**) with strict typing for improved performance. The community encourages developers to port popular JS and TS libraries to the OpenHarmony platform, providing detailed official documentation outlining 79 adaptation rules and modification suggestions for transitioning source code from TS/JS to ArkTS (see specifics in Section 2.1), along with four library porting specifications for guidance (see specifics in Section 2.2). Straying from these adaptation rules may trigger errors or warnings when running TS/JS code on the OpenHarmony platform, leading to compilation failures for ArkTS code. However, our prior examination of 100 renowned software libraries in the *npm* central repository revealed that, on average, each library necessitates developers to rectify 687.5 mismatches of ArkTS syntax as reported by the *ArkCompiler* [3]. This adaptation process demands a thorough grasp of ArkTS syntax, adherence to porting specifications, and substantial manual modification efforts. Thus, an effective technique is urgently needed to automate the porting of TS/JS libraries to the OpenHarmony platform, supporting a vibrant software supply chain.

As a new programming language, ArkTS currently lacks essential tools for automated analysis and porting of software libraries. Nonetheless, the rise of Large Language Models (LLMs), proven in code generation and repair, shows promise for effectively addressing automated porting tasks. Yet, using LLMs to automate the porting of TS/JS libraries to OpenHarmony presents two challenges:

- ***Challenge 1:*** *LLMs struggle to understand the syntactical differences between ArkTS and JS/TS, as well as the diverse adaptation scenarios.* Due to the limited availability of ArkTS software and the lack of exposure of popular LLMs to its code, LLMs find it challenging to understand the syntactical differences between ArkTS and JS/TS. Additionally, OpenHarmony's official documentation provides only simple code examples for each adaptation rule, which is insufficient for LLMs to fully grasp the rule descriptions and address various complex adaptation scenarios.
- ***Challenge 2:*** *Project-level code adaptation tasks often require correcting hundreds of syntax mismatches, which poses a challenge to LLMs as they need to manage the interference between various syntax mismatches and interdependent syntax-mismatching code.* Porting a TS/JS library to OpenHarmony is a project-level code adaptation task. Source files may have hundreds of mismatches with ArkTS syntax rules, and multiple syntax mismatches often occur simultaneously within a single code block. In fact, different syntax mismatches can interfere with one another, and interdependent syntax-mismatching code can affect one another. For a software library, resolving

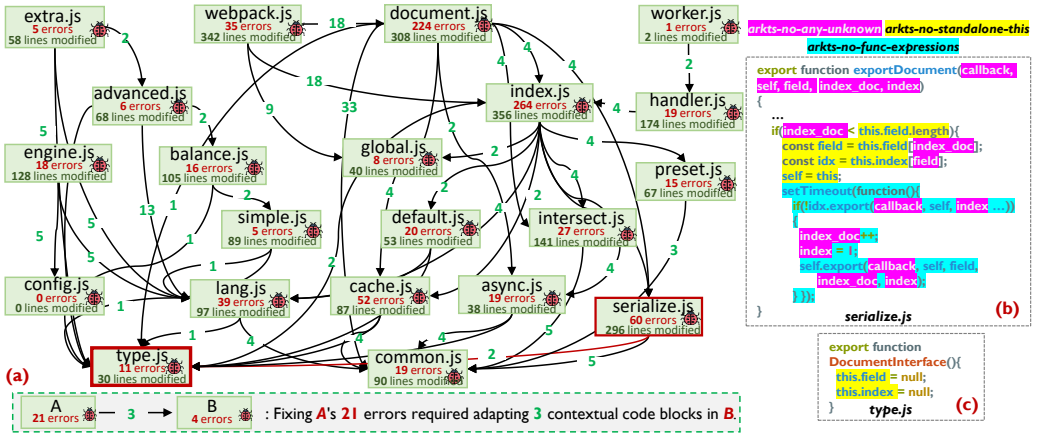


Fig. 1. An example illustrating the challenges associated with project-level code adaptation tasks

any syntax mismatch may introduce new issues in its contextual code, which can lead LLMs into an iterative adaptation process, increasing the complexity of the task.

Figure 1 illustrates the challenges associated with project-level code adaptation tasks. We analyze the porting record of the JS library *flexsearch* in the *OHPM* community. The pre-ported version reported 863 compilation errors as identified by *ArkCompiler* on OpenHarmony. Developers modified 23 source files, adapting a total of 2,569 lines of code. Through our investigation of code adaptation, we identify notable syntactical differences between ArkTS and TS/JS in the definitions and usage of code entities like *Class* and *Module System*. Adapting to these syntax mismatches requires extensive contextual information for variable type inference and often necessitates restructuring code constructs, resulting in substantial changes to TS/JS code. Specifically, the modifications involve 694 complex changes such as function decomposition, parameter type inference, and class restructuring. Additionally, 88.64% of these syntax mismatches require coordinated changes across the contexts of multiple files.

As shown in Figure 1(a), *serialize.js* (with 60 syntax mismatches) depends on *type.js* (with 11 syntax mismatches). During the adaptation process, the interdependent syntax-mismatching code can influence each other. If the developer first resolves the 60 syntax mismatches in *serialize.js*, it may result in *serialize.js* referencing syntax-mismatching code from *type.js*. Later, when adapting *type.js*, any changes made (for instance, the *DocumentInterface()* function in Figure 1(c) needs to be restructured into a class due to the use of the `this` keyword, which mismatches the *arkts-no-standalone-this* syntax rule) can introduce new issues in dependent files like *serialize.js*. This can trap the LLM in an iterative adaptation process, increasing the complexity of the task.

Figure 1(b) shows how different syntax mismatches interfere in a code block. For example, the *exportDocument(callback, self, field, index_doc, index)* function in *serialize.js* does not declare parameter types, uses the `this` keyword, and calls *setTimeout()* with a function expression. This mismatches three ArkTS syntax rules: *arkts-no-any-unknown*, *arkts-no-standalone-this*, and *arkts-no-func-expressions*. To follow the *arkts-no-standalone-this* rule, the function must be restructured into a class, which requires clear parameter type definitions. Additionally, the function expression relies on parameter types; incorrect types can violate the *arkts-no-func-expressions* rule. Thus, fixing these two issues depends on the resolution of *arkts-no-any-unknown* rule. Without an effective strategy, LLMs struggle to manage the interference between various syntax mismatches.

To tackle these challenges, we introduce `ArkAdapter`, an automated code adaptation technique at the project level. For ***Challenge 1***, `ArkAdapter` creates an adaptation knowledge repository that

helps LLMs understand the syntax differences and adaptation rules between ArkTS and TS/JS using retrieval-augmented generation (RAG). More importantly, we expanded a set of real code adaptation examples covering various scenarios, drawing on expert experience to improve LLMs' ability to fix syntax mismatches through few-shot learning. Specifically, we identified common combinations of syntax mismatches that often occur in the same code block and enriched our collection of adaptation instances across different programming paradigms to help LLMs grasp these complex scenarios. For ***Challenge 2***, our `ArkAdapter` prioritizes adaptations for TS/JS libraries with numerous mismatches to ArkTS syntax rules based on five criteria: (1) Test code takes precedence over project code; (2) Prioritize fixing mismatches caused by previous adaptations; (3) Adaptation is done from the bottom up, considering dependency relationships among mismatched code; (4) Prioritize adaptation based on the impact scope of a syntax mismatch (see Table 1: *single-line* > *single-hunk* > *multi-hunk*); (5) Prioritize adaptation based on the granularity of code modifications (see Table 1: *Type System* > *Variable Declaration* > *Object Mechanism* > *Expression Statement* > *Function Declaration* > *Interface Declaration* > *Class Declaration* > *Module System*). When the five criteria conflict, they should be prioritized in the order of (1) to (5). This prioritization helps minimize interference among syntax mismatches and interdependent code, reducing the risk of introducing new issues.

To evaluate `ArkAdapter`, we first construct a high-quality benchmark including 209 pairs of source files with actual code adaptation records performed by OpenHarmony developers. By comparing the pre-ported TS/JS file and post-ported ArkTS file, we found that (1) `ArkAdapter` achieves a *Precision* of 86.84% and a *Recall* of 100.0% on syntax-mismatching code location. (2) 80.14% code adaptations inferred by `ArkAdapter` with *DeepSeek Coder* exactly or plausibly match the actual adaptations made by OpenHarmony developers. Moreover, we utilize `ArkAdapter` to port 79 popular TS/JS libraries to the *OHPM* central repository for validation by official reviewers. Of these, **70 libraries passed validation by *Arkcompiler* and test cases, receiving approval from *OHPM* central repository reviewers.** All uploaded ArkTS libraries scored highly in the *OHPM* central repository evaluations, showing compliance with community porting specifications. The positive feedback highlights the effectiveness of our porting approach.

In summary, the main contributions of this paper are as follows:

- ***An effective code adaptation tool.*** We developed a project-level code adaptation tool `ArkAdapter` to help developers automatically port TS/JS libraries to OpenHarmony.
- ***A high-quality benchmark.*** We constructed a high-quality benchmark including 209 pairs of source files with actual code adaptation records. Within the benchmark, on average, each ArkTS file is covered by 5.63±3.61 test cases and incorporates 20.46±20.76 lines of code adaptations.
- ***Thorough comparisons.*** To provide a comprehensive evaluation, we experiment on three popular LLMs and compare `ArkAdapter` with four alternative prompting methods.
- ***Poring real libraries.*** We utilized ArkAdapter to port 79 widely-used TS/JS libraries. 70 ArkTS libraries (88.6%) were validated by *ArkCompiler*, passed tests, and were approved by the reviewers of the *OHPM* central repository.

2 Background

2.1 Adaptation Rules from TypeScript or JavaScript to ArkTS Language

ArkTS is the main language for OpenHarmony apps. It builds on TypeScript, which adds types to JavaScript. ArkTS has stricter rules than TS to reduce runtime issues and improve efficiency. The OpenHarmony community has created official documentation [1] that lists 79 adaptation rules and suggestions for moving from TS/JS to ArkTS. **Each adaptation rule corresponds to a mismatch of a specific ArkTS syntax rule.** Ignoring these rules can lead to errors or warnings when running TS/JS code on OpenHarmony, and ArkTS code can fail to compile.

Table 1 represents the categorization of 79 adaptation rules and the complexity of resolving syntax mismatches between ArkTS and TS/JS. There are notable syntactical differences between

Table 1. 79 Adaptation Rules from TS/JS to ArkTS language Provided by OpenHarmony

Types	Descriptions of Adaptation Rules
Variable Declaration	●♣(***arkts-no-as-const***) The '*as const*' assertion is not supported;●♣(***arkts-no-any-unknown***) Use specific types instead of 'any' or 'unknown'; ●♣(***arkts-no-props-by-index***) Accessing fields by index is not supported;●(***arkts-no-var***) Use '*let*' instead of '*var*';●(***arkts-no-destruct-assignment***) Destructuring assignment is not supported;●(***arkts-no-destruct-decls***)Destructuring variable declaration is not supported; ●♣(***arkts-unique-names***) Names of types and namespaces must be unique;●♠(***arkts-no-aliases-by-index***) Index access type is not supported.
Function Declaration	● (***arkts-no-destruct-params***) Function declaration with parameter destructuring is not supported; ● (***arkts-no-func-props***) Modifying function declaration properties is not supported;■(***arkts-no-generators***) Generator function is not supported; ●♣(***arkts-no-implicit-return-types***) Omitting function return type annotations is restricted;●♠♣(***arkts-no-inferred-generic-params***) Explicitly annotate generic function type arguments;■♣(***arkts-no-nested-funcs***)Declaring functions inside functions is not supported; ●♣(***arkts-no-standalone-this***) Using '*this*' in functions and static methods of classes is not supported;■♣(***arkts-no-func-expressions***) Use arrow functions instead of function expressions; ●♣(***arkts-no-ctor-signatures-funcs***) Constructor type is not supported
Class Declaration	■♣(***arkts-extends-only-class***) Interfaces cannot extend classes;■(***arkts-implements-only-iface***) Classes do not allow '*implements*'; ■♠♣(***arkts-no-ctor-prop-decls***) Declaring fields in the constructor is not supported; ●♣(***arkts-no-private-identifiers***)Private field starting with # is not supported;■(***arkts-no-class-literals***) Class expression is not supported; ●(***arkts-no-classes-as-obj***)The '*class*' keyword cannot be used as an object;■♣(***arkts-no-ctor-signatures-type***) Use '*class*' instead of types with constructor signatures; ●♣(***arkts-no-call-signatures***) Use '*class*' instead of types with call signatures
Interface Declaration	●♣(***arkts-no-ctor-signatures-iface***) Constructor signatures is not supported in interfaces;■♣(***arkts-no-extend-same-prop***) Interface cannot inherit two interfaces that have the same method
Module System	●(***arkts-limited-stdlib***) Restrictions on using the standard library;●(***arkts-no-umd***) Universal Module Definition is not supported; ■(***arkts-no-ambient-decls***) Ambient module declaration is not supported;■(***arkts-no-func-bind***) '*Function.bind*' is not supported ●♣(***arkts-no-export-assignment***) The '*export = · · ·* ' syntax is not supported;■(***arkts-no-func-apply-call***) '*Function.apply*' and '*Function.call*' are not supported; ●(***arkts-no-globalthis***) '*globalThis*' is not supported;●(***arkts-no-import-assertions***) Import assertion is not supported; ●(***arkts-no-module-wildcards***) Wildcard is not supported in module names;●(***arkts-no-ts-deps***) *.ets files are allowed to import source code from *.ets/*.ts/*.js files, but *.ts/*.js files are not allowed to import source code from *.ets files●♣(***arkts-no-require***)Assignment expression with require and import is not supported;■(***arkts-no-misplaced-imports***) Only import statements are allowed before import statements; ●(***arkts-no-import-default-as***)'import default as ...' is not supported
Object Mechanism	●(***arkts-no-method-reassignment***)Modifying object methods is not supported;■♣(***arkts-no-prototype-assignment***) Assigning values on the prototype is not supported; ●♠♣(***arkts-no-ns-as-obj***) Namespaces cannot be used as objects; ●(***arkts-no-new-target***)'*new.target*' is not supported ●(***arkts-identifiers-as-prop-names***) Property names of objects must be valid identifiers
Type System	●♠♣(***arkts-no-conditional-types***) Conditional type is not supported ●♣(***arkts-no-obj-literals-as-types***)Object literals cannot be used for type declarations;●(***arkts-no-definite-assignment***) Definite assignment assertion is not supported; ●♣(***arkts-no-indexed-signatures***) Index signature is not supported; ●♠♣(***arkts-no-mapped-types***) Mapped type is not supported; ●(***arkts-no-polymorphic-unops***) Unary operators +, -, and are only applicable to numeric types; ●♣(***arkts-no-types-in-catch***)Type annotations are not supported in catch clauses; ●♣(***arkts-no-typing-with-this***)The 'this' type is not supported; ●(***arkts-no-utility-types***) Some utility types are not supported; ●♣(***arkts-strict-typing***) Strict type checking is enforced ●(***arkts-no-structural-typing***) Structural typing is not supported; ●♣(***arkts-no-noninferrable-arr-literals***) Array literals must only contain elements of inferable types; ●♣(***arkts-no-untyped-obj-literals***) Explicitly annotate the type of object literals; ●♠♣(***arkts-no-enum-mixed-types***) Enum members must be initialized with compile-time expressions of the same type
Expression Statement	★(***arkts-no-multiple-static-blocks***) Only one static block is supported; ●(***arkts-limited-esobj***) Restrictions on using the ESObject type; ●(***arkts-no-jsx***)JSX expressions are not supported; ●♠♣(***arkts-as-casts***)Type casting only supports the 'as T' syntax; ●♣(***arkts-limited-throw***) Restrictions on the types of expressions in throw statements;■♣(***arkts-no-for-in***) '*for ... In*' is not supported; ●♠♣(***arkts-no-ns-statements***) Non-declaration statements are not supported in namespaces; ■♣(***arkts-no-with***) The 'with' statement is not supported; ★♠♣(***arkts-no-enum-merging***) Enum declaration merging is not supported; ★♣(***arkts-no-decl-merging***) Declaration merging is not supported; ●♣(***arkts-strict-typing-required***) Disabling type checking through comments is not allowed
Operator	●(***arkts-instanceof-ref-types***) Partial support for the '*instanceof*' operator; ●♠♣(***arkts-no-is***) Use '*instanceof*' and '*as*' for type guarding●(***arkts-no-in***)The '*in*' operator is not supported●(***arkts-no-spread***)Partial support for the spread operator ●♠♣(***arkts-no-intersection-types***) Use inheritance instead of intersection types ●♣(***arkts-no-comma-outside-loops***) The commaoperator is only used in for loop statements;●(***arkts-no-symbol***) The *Symbol()* API is not supported ●♣(***arkts-no-type-query***)The '*typeof*' operator is only allowed in expressions ●(***arkts-no-delete***)The '*delete*' operator is not supported

Each rule is presented following the format: (***error message reported by ArkCompiler***) *Description for the adaptation rule.*
●, ■ and ★ represent the adaptation rules that pertain to syntax mismatches in ***single-line***, ***single-hunk*** and ***multi-hunk*** code.
♣ represent the adaptation rules that constrain JavaScript's Non-standard *ES6+ Coding Standards* in ArkTS syntax.
♠ represent the rules that adapt TypeScript's advanced types. Each adaptation rule corresponds to a mismatch of a specific ArkTS rule.

ArkTS and TS/JS in the definitions and usage of code entities such as *Module System*, *Type System*, *Expression Statement*, etc. As shown in Table 1, 12 out of 79 rules are used to adapt the TS's complex data structures, such as ***arkts-no-conditional-types*** (which does not support conditional types) and ***arkts-no-mapped-types*** (which does not support mapped types). 43 out of 79 rules constrain JavaScript's Non-standard *ES6+ Coding Standards* in ArkTS syntax, such as ***arkts-no-func-expressions*** (which prohibits function expressions). Adapting to these syntax mismatches not only requires extensive contextual information for type inference of variables but also involves the restructuring of code constructs such as classes, interfaces, and functions, leading to substantial code changes for TS/JS code. Specifically, there are 61, 15, and 3 rules concerning syntax mismatches in *single-line*, *single-hunk*, and *multi-hunk* code, respectively. 89% of syntax-mismatching code require modifications at the multi-hunk level. Additionally, each code line or code hunk may mismatch multiple ArkTS syntax rules simultaneously, intensifying the complexity of the adaptation process.

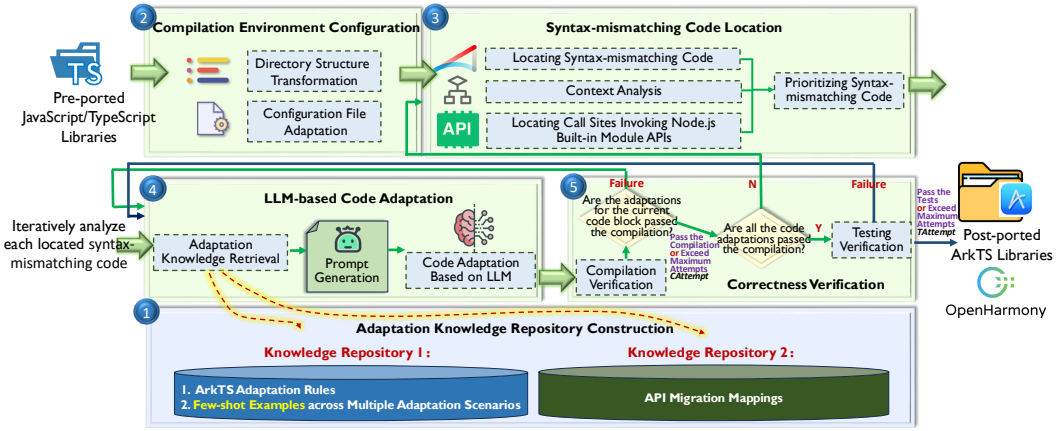


Fig. 2. The overall architecture of ArkAdapter

For a software library, addressing any syntax mismatch may potentially induce new issues to its contextual code, increasing the complexity of the adaptation task.

2.2 Library Porting Specifications Defined by OpenHarmony Community

The OpenHarmony community recommends developers prioritize porting high-quality popular TS/JS libraries and adhere to the following four standard guidelines [14]:

- ***Specification 1:*** *The OpenHarmony development framework does not support Node.js built-in modules.* Before porting, it is advisable to utilize the `js-e2e` tool [12] to identify the list of Node.js built-in module APIs that the software library relies on. Then, find equivalent APIs in the ArkTS built-in modules to replace the Node.js functionalities.
- ***Specification 2:*** *The ArkCompiler is compatible with a specific software directory structure and dependency configuration format known as* `oh-package.json5`. It is crucial to align the directory structure and original `package.json` of TS/JS libraries accordingly. Specifically, when porting a software library, it is vital to verify that its dependencies as configured in the `package.json` have been successfully migrated.
- ***Specification 3:*** *The runtime environment of the OpenHarmony platform is incompatible with some syntax features of TS/JS.* To address this, it is advisable to copy the source code files to the `src/main/ets` directory and, following the adaptation guidelines detailed in Section 2.1, convert the TS/JS code to the ArkTS language.
- ***Specification 4:*** *Adapting the test cases of TS/JS libraries is essential.* Verify that the ported ArkTS library passes these test cases and produces outcomes consistent with the pre-ported version. To achieve this, it is advisable to move the test code to the `src/ohosTest/ets` directory. Refer to the rules for converting from TS/JS to ArkTS and use the test assertion interfaces provided by OpenHarmony [14] to ensure successful test adaptation.

3 ArkAdapter Approach

Figure 2 depicts the architecture of `ArkAdapter`. It establishes an adaptation knowledge repository for LLMs to understand syntax differences between TS/JS and ArkTS using retrieval-augmented generation (RAG). Real code adaptation instances were expanded to enhance LLMs' ability in fixing syntax mismatches through few-shot learning. `ArkAdapter` prioritizes code adaptations by considering factors like dependency structures and syntax mismatch granularity, which helps prevent interference among various syntax mismatches and their interdependent code. It incorporates context analysis to modify relevant contextual code during adaptation and automates the porting process from TS/JS to ArkTS in five key steps.

3.1 Step 1: Adaptation Knowledge Repository Construction

The `ArkAdapter` uses RAG to enhance LLM for code adaptation. Two knowledge repositories are in place: (1) *ArkTS adaptation Rules Repository* with rule descriptions and few-shot examples. (2) *API Migration Mappings Repository* documenting Node.js to ArkTS built-in module API migrations and test assertion mappings to OpenHarmony.

3.1.1 Knowledge Repository of ArkTS Adaptation Rules. According to the 79 adaptation rules provided in the official documentation for converting from TS/JS to ArkTS Language [1], we represent each adaptation rule in the knowledge repository as a quadruple: *Rule* = <*Error*, *Description*, *Suggestions*, *Few-shot adaptation examples*>, where:

- *Error*: The error/warning message reported by *ArkCompiler* for a specific ArkTS syntax rule mismatch (e.g., ***arkts-no-as-const***) corresponds to an adaptation rule, as shown in Table 1.
- *Description*: A natural language explanation of a specific ArkTS syntax rule mismatch outlined in the official documentation [1].
- *Suggestions*: A natural language explanation of the adaptation suggestions corresponding to a specific ArkTS syntax rule mismatch in the official documentation [1].
- *Few-shot adaptation examples*: Few-shot adaptation examples, empirically enriched by `ArkAdapter` across diverse scenarios, empower our tool to utilize few-shot learning effectively, enhancing LLM's comprehension of code adaptations in varied contexts.

Empirically Enriching Code Adaptation Examples Across Diverse Scenarios. To enhance the supportive effect of few-shot learning on LLMs, we perform the following three tasks to enrich high-quality code adaptation instances:

- ***Task 1 Collecting real instances of syntax-mismatching code:*** We collect the top 100 most downloaded TS/JS popular libraries from the *npm* repository and utilize *ArkCompiler* to identify and report mismatch points of ArkTS syntax rules. The error messages from the compiler (e.g., ***arkts-no-as-const***) assist us in directly mapping to the corresponding code adaptation rules. It is crucial to note that *ArkCompiler* can only pinpoint the starting code line where a syntax mismatch occurs. To facilitate a comprehensive context analysis, we extracted complete function code blocks containing the problematic lines. Through this approach, we gathered a total of 5,934 code blocks that resulted in 214,626 compilation errors.
- ***Task 2 Selecting syntax-mismatching code instances across various scenarios:*** In practice, each code line/hunk may mismatch one syntax rule or simultaneously mismatch multiple syntax rules. Therefore, we empirically enrich real code adaptation instances for scenarios involving both "*mismatch of a single syntax rule*" and "*mismatch of multiple syntax rules*":
  - ***Mismatch of A Single Syntax Rule.*** A total of 1,972 code blocks result in a single compilation error, encompassing mismatches of 79 syntax rules. In this study, two authors with three years of TS/JS development experience individually analyzed and identified code blocks with contextual variations for each mismatched syntax rule to cover a wide range of adaptation scenarios. Specifically, concerning the 17 syntax rules that involve four programming paradigms in TS/JS, our collection meets four criteria:
    (1) *Imperative Programming*: For the syntax rules ***arkts-no-destruct-assignment*** and ***arkts-no-for-in***, our gathered code blocks covering four implementation scenarios of *imperative programming*: *sequential control*, *conditional control*, *looping control*, and *state management*.
    (2) *Functional Programming*: Concerning the syntax rules ***arkts-no-destruct-params***, ***arkts-no-implicit-return-types***, ***arkts-no-nested-funcs***, ***arkts-no-func-expressions***, and ***arkts-no-ctor-signatures-funcs***, our collected code blocks strive to cover four implementation scenarios of *functional programming*: *pure functions*, *higher-order functions*, *function composition*, and *immutable data*.

	common combinations of syntax mismatches (*mismatch*$_1$, *mismatch*$_2$,..., *mismatch*$_n$, occurrences)		
n=2	(*arkts-no-func-expressions*, *arkts-no-var*, 13) (*arkts-no-any-unknown*, *arkts-no-inferred-generic-params*, 14) (*arkts-no-any-unknown*, *arkts-no-standalone-this*, 19) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, 55) (*arkts-no-any-unknown*, *arkts-no-untyped-obj-literals*, 57) (*arkts-no-func-expressions*, *arkts-no-standalone-this*, 71) (*arkts-no-var*, *arkts-no-untyped-obj-literals*, 109) (*arkts-no-any-unknown*, *arkts-no-var*, 274) (*arkts-no-any-unknown*, *arkts-no-implicit-return-types*, 301)	n=3	(*arkts-no-func-expressions*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, 14) (*arkts-no-func-expressions*, *arkts-no-standalone-this*, *arkts-no-prototype-assignment*, 17) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-standalone-this*, 19) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-untyped-obj-literals*, 20) (*arkts-no-any-unknown*, *arkts-no-comma-outside-loops*, *arkts-no-implicit-return-types*, 21) (*arkts-no-any-unknown*, *arkts-no-implicit-return-types*, *arkts-no-untyped-obj-literals*, 27) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-implicit-return-types*, 57) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-var*, 61) (*arkts-no-var*, *arkts-no-any-unknown*, *arkts-no-implicit-return-types*, 71)
n=4	(*arkts-no-var*, *arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-standalone-this*, 12) (*arkts-no-any-unknown*, *arkts-no-comma-outside-loops*, *arkts-no-implicit-return-types*, *arkts-no-var*, 13) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-untyped-obj-literals*, *arkts-no-inferred-generic-params*, 14) (*arkts-no-var*, *arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-implicit-return-types*, 15) (*arkts-no-any-unknown*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, *arkts-no-func-apply-call*, 15) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, 16) (*arkts-limited-stdlib*, *arkts-no-func-expressions*, *arkts-no-implicit-return-types*, *arkts-no-untyped-obj-literals*, 21) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-nested-funcs*, *arkts-no-untyped-obj-literals*, 23) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-implicit-return-types*, *arkts-no-var*, 41) (*arkts-no-any-unknown*, *arkts-no-func-expressions*, *arkts-no-standalone-this*, *arkts-no-prototype-assignment*, 62)		
n=5	(*arkts-no-var*, *arkts-no-func-expressions*, *arkts-no-any-unknown*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, 18) (*arkts-no-func-props*, *arkts-no-func-expressions*, *arkts-no-method-reassignment*, *arkts-no-any-unknown*, *arkts-no-implicit-return-types*, 22) (*arkts-no-var*, *arkts-no-func-expressions*, *arkts-no-any-unknown*, *arkts-no-standalone-this*, *arkts-no-prototype-assignment*, 52) (*arkts-no-func-expressions*, *arkts-no-any-unknown*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, *arkts-no-prototype-assignment*, 63)		
n=6	(*arkts-no-func-expressions*, *arkts-no-any-unknown*, *arkts-no-standalone*, *arkts-no-implicit-return-types*, *arkts-no-polymorphic-unops*, *arkts-no-prototype-assignment*, 40) (*arkts-no-var*, *arkts-no-func-expressions*, *arkts-no-any-unknown*, *arkts-no-standalone-this*, *arkts-no-implicit-return-types*, *arkts-no-prototype-assignment*, 63)		

Fig. 3. Combinations of Syntax Mismatches that Frequently Co-occur within the Same Code Blocks

(3) *Object-Oriented Programming*: For the ten syntax rules ***arkts-no-props-by-index***, ***arkts-no-standalone-this***, ***arkts-extends-only-class***, ***arkts-implements-only-iface***, ***arkts-no-class-literals***, ***arkts-no-classes-as-obj***, ***arkts-no-method-reassignment***, ***arkts-no-prototype-assignment***, ***arkts-no-ns-as-obj***, and ***arkts-no-intersection-types***, our code collection is intended to cover two implementation scenarios: *inheritance* and *polymorphism*.
(4) *Asynchronous Programming*: Regarding the syntax rule ***arkts-no-generators***, our collected code blocks need to cover three classic implementation scenarios of *asynchronous programming*: *callback functions*, *Promises*, and *async/await* syntax.
The two authors independently analyze and select code blocks covering multiple implementation scenarios. Through cross-validation, the inter-rater agreement, as measured by *Cohen's kappa*, stands at 0.92, signaling a near-perfect agreement between the raters [31].

- ***Mismatch of Multiple Syntax Rules.*** 3,962 code blocks introduce multiple compilation errors simultaneously. As shown in Figure 3, we identify 34 combinations of syntax mismatches that frequently co-occur within the same code blocks (with occurrences exceeding the average frequency). We retain 1,710 code blocks that trigger the common combinations of syntax mismatches.

- ***Task 3 Acquiring a range of code adaptation instances by leveraging developers' practical experience:*** The two authors of our paper who having three years of TS/JS software development experience, manually adapt the 925 code blocks according to the adaptation rules provided by OpenHarmony. They cross-validated the correctness of code adaptation (*Cohen's kappa* = 0.89) and retain the 836 adaptation instances that passed the compilation in our knowledge repository.

3.1.2 Knowledge Repository of API Migration Mappings. As outlined in the library porting specifications by OpenHarmony [14], our task involves exploring: (1) *API migration mappings from Node.js built-in modules to ArkTS built-in modules*, as well as (2) *API migration mappings for testing assertions from Node.js to the OpenHarmony platform*. Fortunately, the migration mappings for testing assertion APIs on the OpenHarmony platform are already documented in the official specifications [14], and we have directly integrated them into our knowledge repository.

Mining API migration mappings from Node.js built-in modules to ArkTS built-in modules. For API migration mappings between Node.js built-in modules and ArkTS built-in modules, we utilize the state-of-the-art tool KGE4A [30] for exploration. KGE4A is a documentation-based approach using knowledge graph embedding for analogical API recommendation. We employ KGE4A to construct an API knowledge graph for ArkTS built-in modules (API level 12) from their API

documentations, leveraging graph embedding to represent nodes and edges as numeric vectors. Given an API from Node.js built-in modules, KGE4A retrieves the most similar ArkTS API from the embedded knowledge graph. We examine 3,027 built-in module APIs from Node.js version 22.9.0, utilizing KGE4A to search for functionally equivalent API mappings within ArkTS built-in modules. We configure KGE4A to provide the top 10 ArkTS APIs most resembling the Node.js APIs. Two authors with three years of TS/JS development experience manually validate API mapping relationships where parameter and return value types can be interchangeably substituted (*Cohen's kappa* = 0.95). Finally, we identify 715 API migration mappings.

3.2 Step 2: Compilation Environment Configuration

According to OpenHarmony's porting specifications, ArkAdapter adjusts the directory structure of TS/JS libraries to meet ArkTS compatibility. It organizes dependency files, License.txt, and README in the project root. The source code is moved to `src/main/ets` and test files to `src/ohosTest/ets`. The source code is converted from *.js/*.ts to *.ets format. For package.json, ArkAdapter aligns fields with those in oh-package.json5 to meet ArkCompiler's requirements. It also ensures that third-party dependencies in oh-package.json5 are migrated to the *OHPM* central repository.

3.3 Step 3: Syntax-mismatching Code Location

For the configured TS/JS library, ArkAdapter identifies syntax-mismatching code and related contextual code. This information is crucial for LLMs to generate effective code adaptations.

Locating Syntax-mismatching Code. First, ArkAdapter uses *ArkCompiler* to compile the source code of the TS/JS library in order to identify mismatches of ArkTS syntax rules. It then employs the js-e2e tool [12] to scan the code and locate the call sites that invoke Node.js built-in module APIs. The above process outputs a collection of syntax-mismatching code *CS*. Each of them $C_i \in CS$ is denoted as a triple <*Error*, *Problematic code block*, *Node.js built-in module API*>.

- *Error:* Error/warning messages (e.g., ***arkts-no-as-const***) reported by *ArkCompiler* corresponding to the mismatches of ArkTS syntax rules.
- *Problematic code blocks: ArkCompiler* only reports the starting line number of the problematic code. After categorizing and analyzing the 79 adaptation rules in Table 1, we found that 61, 15, and 3 rules pertain to syntax mismatches in *single-line*, *single-hunk*, and *multi-hunk code*, respectively. In cases of syntax mismatches occurring in *single-line* or *single-hunk* code, ArkAdapter identifies the problematic code block by returning the function body containing the compiler error line. As for the three types of syntax mismatches involving *multi-hunk* code, ArkAdapter performs the following tasks to identify problematic code blocks: (1) returning multiple function bodies within the same class file that trigger the error message ***arkts-no-multiple-static-blocks***; (2) finding multiple code blocks that raise the error message ***arkts-no-enum-merging*** and contain multiple identically named enum definitions; (3) locating code blocks that trigger the error ***arkts-no-decl-merging*** and declare class/interface with identical names.
- *Node.js built-in module API or test assertion API:* If the problematic code depends on Node.js built-in modules or test assertions, ArkAdapter records the invoked API signatures to be migrated.

Context Analysis of syntax-mismatching Code. With the assistance of the static analysis framework *ArkAnalyzer* [2] tailored for ArkTS applications provided by OpenHarmony, ArkAdapter analyzes the abstract syntax tree (AST) of TS/JS source code to pinpoint two types of contextual information related to syntax-mismatching code:

- *Contextual code on which the syntax-mismatching code depends.* ArkAdapter identifies the variables, functions, and objects referenced by the syntax-mismatching code and locates their declarations within the AST.

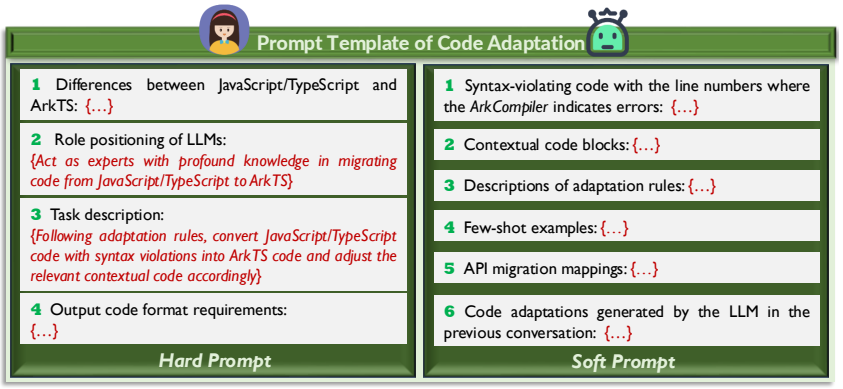


Fig. 4. Prompt Template of Code adaptation

- *Contextual code that depends on the syntax-mismatching code.* Utilize the *ArkAnalyzer* API *findReferencesAsNodes()* to locate the nodes that invoke the syntax-mismatching code in the AST, and retrieve the code blocks that implement these nodes.

Adaptation Priority Strategy. In a TS/JS library undergoing migration, it is common to encounter hundreds of syntax-mismatching code blocks. Moreover, a single code block may mismatch multiple syntax rules. Adapting a code block could reintroduce new issues into previously adapted code, initiating iterative adaptation cycles. To streamline this process, `ArkAdapter` prioritizes syntax-mismatching code by considering both the dependency structure and the granularity of syntax-mismatching code. The adaptation priority strategy is outlined below:

- ***Overall Adaptation Priority.*** `ArkAdapter` considers three levels of prioritization: (1) *Test code takes precedence over project code.* This allows test validation for code adaptations. (2) *Prioritize addressing syntax mismatches introduced by the preceding adaptation operation.* This approach facilitates easier resolution of newly introduced problems while maintaining the model's memory of the previous adaptation. (3) *Adaptation is carried out from the bottom up based on the call graph structure of the syntax-mismatching code.* This entails prioritizing the adaptation of problematic code blocks that do not depend on other functions, followed by adapting code blocks where all dependent functions have already been adapted. This prioritization helps minimize interference among interdependent syntax-mismatching code, reducing the risk of introducing new issues.
- ***Code Granularity-based Adaptation Priority.*** (4) `ArkAdapter` prioritizes adaptation based on the impact scope of a syntax mismatch, ranging from the smallest to the largest: *single-line* > *single-hunk* > *multi-hunk*. (5) `ArkAdapter` prioritizes the multiple syntax mismatches that co-occur in a code block according to the granularity of code modification, from the smallest to the largest as outlined in Table 1: *Type System* > *Variable Declaration* > *Object Mechanism* > *Expression Statement* > *Function Declaration* > *Interface Declaration* > *Class Declaration* > *Module System*. This approach helps prevent interference among multiple syntax mismatches.
  When the five criteria conflict, they should be prioritized in the order of (1) to (5).

3.4 Step 4: LLM-based Code adaptation

The `ArkAdapter` leverages the code understanding and repair capabilities of LLMs to generate adaptation AC_i for syntax-mismatching code $C_i \in CS$ in a prioritized sequence. Figure 4 outlines the prompt template provided to LLMs, which consists of two main components:

- ***Hard Prompt***: (1) *Introducing the differences between TS/JS and ArkTS languages*; (2) *Role positioning of LLMs*: Enabling LLMs to act as experts with profound knowledge in migrating code from TS/JS to ArkTS. (3) *Task description*: Following adaptation rules, convert the TS/JS code with syntax mismatches into ArkTS code; and adjust the relevant contextual code accordingly. (4) *Output code format requirements.*

- ***Soft Prompt***: (1) *syntax-mismatching code with the line numbers where the ArkCompiler indicates errors;* (2) *Contextual code blocks of the syntax-mismatching code;* (3) *Descriptions of adaptation rules for the syntax mismatches sourced from our knowledge repository using error messages;* (4) *Few-shot examples extracted by* `ArkAdapter` *illustrating each adaptation rule;* (5) *ArkTS APIs that can serve as replacements for Node.js built-in module APIs or test assertions, retrieved from our knowledge repository.* (6) In scenarios where the current analysis of syntax-mismatching code arises from issues introduced in the prior adaptation steps, *the code generated by the LLM in the previous conversation* should also be included as one of the input elements.
  In our knowledge repository, we empirically expanded an average of 19.23 ± 5.97 adaptation instances for each ArkTS syntax rule across various implementation scenarios. Following the approach[42], due to the LLM's input constraints, we utilize two few-shot examples closely resembling the syntax-mismatching code block by retrieving them from the knowledge repository using the *cosine similarity metric*.

3.5 Step 5: Correctness Verification

For the adaptation AC_i derived from LLM for code block $C_i \in CS$, `ArkAdapter` uses *ArkCompiler* to validate the compilation. If AC_i fails this validation, the tool returns to **Step 4** to try adapting C_i again, with a maximum number of attempts defined by $CAttempt$.

If AC_i compiles successfully, `ArkAdapter` checks if all code blocks in the set CS are adapted without errors. If so, it tests the adapted ArkTS code. We emphasize that our tool not only verifies whether the adapted code passes tests, but also compares the test outcomes before and after porting using differential testing to ensure consistency. If any code blocks fail the tests, `ArkAdapter` logs those failures and returns to **Step 4** for further adaptation, limited by $TAttempt$.

If there are still unadapted code blocks in CS, the process loops back to **Step 3** for more analysis and adaptation. The process stops under two conditions: (1) All code blocks in CS have been attempted for adaptation, and the maximum attempts for those that failed compilation or testing have been reached. (2) All adaptations in CS pass validation, allowing `ArkAdapter` to output the successfully adapted ArkTS Library.

4 Evaluation

We study three research questions in our evaluation section:

- **RQ1 (Effectiveness of Mismatch Points Location)**: ***How effectively can*** `ArkAdapter` ***locate the TS/JS code that mismatches the syntax rules of ArkTS?*** To answer RQ1, leveraging our benchmark, we employ `ArkAdapter` to identify syntax-mismatching code and the corresponding contextual code in the pre-ported TS/JS file, and evaluate its *Precision* and *Recall* of location.
- **RQ2 (Effectiveness of Code Adaptation)**: ***How effectively can*** `ArkAdapter` ***adapt the non-compliant TS/JS code into ArkTS code?*** To answer RQ2, we evaluate the effectiveness of `ArkAdapter` in code adaptation by comparing the similarity between its inferred code adaptations and the actual adaptations carried out by OpenHarmony developers.
- **RQ3 (Usefulness of** `ArkAdapter`**)**: ***Can*** `ArkAdapter` ***reliably port real-world TS/JS libraries to OpenHarmony?*** To answer RQ3, We use `ArkAdapter` to port 79 popular TS/JS libraries and submit them to the *OHPM* central repository for official review. We assess the usefulness of `ArkAdapter` based on feedback received during the validation process.

4.1 Data Collection

We collect the pre-ported and post-ported versions of software libraries from the OHPM central repository and use code comparisons to create accurate records of porting adaptations made by OpenHarmony developers. This collection serves as a benchmark to evaluate the effectiveness of `ArkAdapter`. The benchmark construction involves four steps:

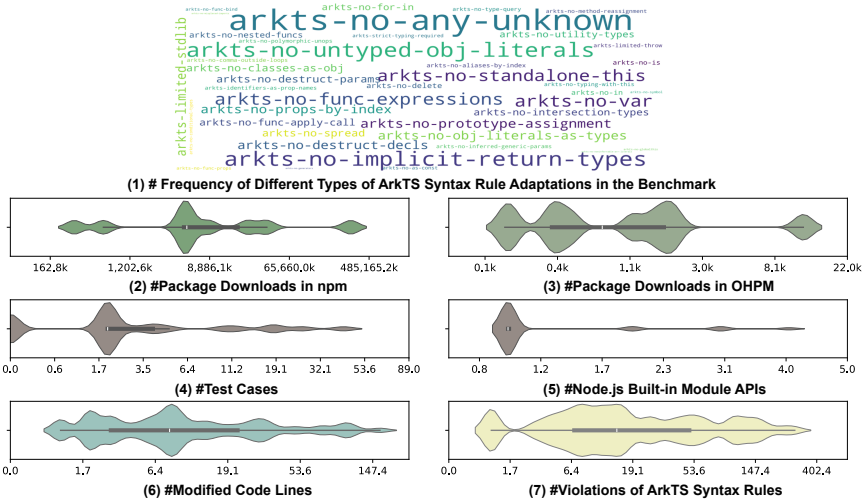


Fig. 5. Demographics of Our Collected Benchmark

- As of October 1, 2024, the OHPM central repository has aggregated 839 third-party libraries (including ArkTS, JavaScript, TypeScript, and C/C++ libraries). After thoroughly analyzing all 93 ArkTS libraries currently in the OHPM, we retained 19 libraries that explicitly mentioned the original TS/JS source code repositories in their `README` files.
- We compared the code variances between the pre-ported and post-ported versions of the 19 libraries, collecting pairs of source code files with identical fully qualified name in both versions that exhibit differences in implementation. In total, we identified 209 file pairs, each denoted as $Pair_i =< File_{Pre}, File_{Post} >$, where $File_{Pre}$ represents the pre-ported TS/JS file and $File_{Post}$ denotes its corresponding post-pored ArkTS file.
- Two authors with three years of experience in JavaScript and TypeScript development thoroughly studied 79 adaptation rules and the porting specifications of OpenHarmony. By comparing each pair of files, they manually annotated code adaptations that rectify syntax mismatches to eliminate interference caused by "new feature addition" or "code refactoring" in our evaluation. Through cross-validation, the inter-rater agreement, measured by *Cohen's kappa*, is 0.89, indicating a near-perfect agreement between the raters [31].
- For the pre-ported file $File_{Pre}$ and the post-pored file $File_{Post}$, we employed the program analysis tool *ArkAnalyzer* to identify test cases from the associated test suite that cover these two files individually. These test cases are used to verify the correctness of the porting.

Figure 5 describes the demographics of our benchmark. The files in the benchmark are sourced from high-quality, popular software libraries. Within the benchmark, on average, each ArkTS file is covered by 5.63 ± 3.61 test cases (Figure 5(4)). Each ArkTS file relies on 1.49 ± 0.76 Node.js built-in module APIs (Figure 5(5)) and incorporates 20.46 ± 20.76 lines of code changes (Figure 5(6)). Validated by *ArkCompiler*, each file involves 39.72 ± 16.01 ArkTS syntax mismatches (Figure 5(7)). Based on the frequency of different types of ArkTS syntax rule adaptations in the benchmark, we present a "word cloud" as shown in Figure 5(1). The findings reveal that our benchmark encompasses adaptations associated with 79 different syntax rules. These statistics underscore the high quality of our benchmark, rendering it well-suited for assessing the effectiveness of `ArkAdapter`.

4.2 Implementation Details

We implemented the core logic of `ArkAdapter` in Python by accessing the LLM API endpoint. Following a previous approach [29], we configured the parameters in the experiment as follows: (1) utilizing a sampling temperature of 1 to generate a diverse set of potential adaptations, and (2)

Table 2. The effectiveness of ArkAdapter in mismatch points identification

	TP	*FP*	*FN*	*Precision*	*Recall*	*F-measure*
Single-line syntax violating code location	1,391	219	0	86.40%	100%	92.70%
Single-hunk syntax violating code location	928	131	0	87.63%	100%	93.41%
multi-hunk syntax violating code location	23	5	0	82.14%	100%	90.20%
Summary	**2,342**	**355**	**0**	**86.84%**	**100%**	**92.95%**

setting the maximum attempts for readapting after compilation failure (*CAttempt*) and the maximum attempts for readapting after test failure (*TAttempt*) to 10. All experiments were conducted on a Linux machine powered by an AMD Ryzen 7 5800 8-Core Processor running at 3.40 GHz with 16GB of RAM. We ran all experiments in RQ2 and RQ3 ten times and calculated the average results to reduce the interference caused by the randomness of LLMs.

4.3 RQ1: Effectiveness of Mismatch Points Location

Study Methodology. For each instance $Pair_i =< File_{Pre}, File_{Post} >$ in our benchmark, we employ ArkAdapter to identify syntax-mismatching code line/hunk and the corresponding contextual code in the pre-ported TS/JS file $File_{Pre}$. We evaluate the effectiveness of location by comparing the located code with the *actual adaptations* documented in the ArkTS file $File_{Post}$.

Evaluation Metrics. We evaluate six metrics: (1) ***True Positive (TP)***: A syntax-mismatching code line/hunk and its context found by ArkAdapter in $File_{Pre}$ was adapted in $File_{Post}$; (2) ***False Positive (FP)***: A syntax-mismatching code line/hunk and its context found by ArkAdapter in $File_{Pre}$ was not adapted in $File_{Post}$; (3) ***False Negative (FN)***: A code line/hunk and its context adopted in $File_{Post}$ was not located by ArkAdapter in $File_{Pre}$.

Based on the above three metrics, we can obtain the *Precison*, *Recall* and *F-measure* as follows: (a) ***Precision*** = *TP* / (*TP* + *FP*); (b) ***Recall***: *TP* / (*TP* + *FN*); (c) ***F-measure***: 2 × *Precision* × *Recall* / (*Precision* + *Recall*). *Precision* evaluates whether ArkAdapter can locate mismatch points precisely. *Recall* evaluates the capability of ArkAdapter in locating all the mismatch points. *F-measure* combines the *Precision* and *Recall* together [37].

Results. Table 2 shows the experiment results of RQ1. ArkAdapter locates mismatch points in TS/JS file with a *Precision* of 86.84 %, a *Recall* of 100% and a *F-measure* of 92.95%. In the 209 file pairs, the tool identified 1,578 syntax-mismatching code blocks along with their 1,119 corresponding contextual code blocks. Developers made actual modifications to 2,342 code blocks. Notably, for the three types of syntax-mismatching code, ArkAdapter did not miss any instances (FN = 0). However, our tool produced false positives for 219 *single-line*, 131 *single-hunk*, and 5 *multi-hunk* syntax-mismatching codes, respectively.

FP Analysis. We examine the 355 false positive instances and identify two primary reasons:

- *Some syntax mismatch instances do not require adaptation in their contextual code (274/355).* To ensure a precise understanding of the contextual information of syntax-mismatching code and to adjust the affected code appropriately, ArkAdapter includes corresponding contextual code in its prompts. For instance, our tool identified 653 contextual code blocks of the single-line syntax-mismatching code. Among these, 143 contextual code blocks do not necessitate modifications, resulting in false positives. In these instances, the adapted code successfully passes both compilation and testing validation.
- *Developers may sometimes be reluctant to adapt compilation warning issues that do not lead to failures (81/355).* Among the 79 adaptation rules, 5 rules appear as warnings rather than causing compilation errors: ***arkts-limited-esobj***, ***arkts-no-classes-as-obj***, ***arkts-no-func-bind***, ***arkts-no-globalthis*** and ***arkts-no-definite-assignment***. Developers may overlook these warning issues, leading to false positives. However, violating these rules could impact program performance on the OpenHarmony platform. For instance, in the *chardet* library [4], there is

code that violates the ***arkts-no-classes-as-obj*** rule, left unadapted by developers. Yet, similar issues in libraries like *flexsearch* [10] and *pinyin4js* [18] were well addressed by developers.

Conclusion: `ArkAdapter` locates mismatch points in TS/JS file with a *Precision* of 86.84%, a *Recall* of 100% and a *F-measure* of 92.95%. Notably, the tool will not miss reporting any syntax-mismatching code that requires adaptation.

4.4 RQ2: Effectiveness of Code Adaptation

Study Methodology. For each instance $Pair_i =< File_{Pre}, File_{Post} >$ in our benchmark, we utilize `ArkAdapter` to perform code adaptations for the TS/JS file $File_{Pre}$. Additionally, we assess the quality of the *inferred adaptations* by comparing them with the *actual adaptations* made by OpenHarmony developers, as documented in the ArkTS file $File_{Post}$.

Baselines. To provide a comprehensive evaluation, we experiment on three popular LLMs and compare `ArkAdapter` with four alternative prompting methods, with details as below:

- ***LLMs:*** We evaluate the performance of `ArkAdapter` on both open-source and closed-source models, including *ChatGPT*, *Qwen* and *DeepSeek Coder*. *ChatGPT* is a popular closed-source LLM developed by *OpenAI*, which show versatile abilities across different fields such as code generation and program repair. We access it through API `gpt-4o-mini` . *Qwen* and *DeepSeek Coder* are the two open-source LLMs that are affordable for us and rank in the top five on the *Big Code Models Leaderboard* (accessed on October 15, 2024) [5]. *Qwen* is developed by *Alibaba*, which provides powerful capabilities to process code tasks. We access its model services through API `qwen2.5-coder-7b-instruct` . *DeepSeek Coder* is a recent powerful open-source LLM developed by *DeepSeek AI*, which significantly outperforms existing open-source code LLMs. We also access it based on its official API `DeepSeek-V2.5-Chat` .
- ***Alternative Prompting Methods:*** We provide six components for the prompts, including: (1) the identified syntax-mismatching code and the code line numbers where the *ArkCompiler* indicates errors; (2) contextual code blocks of the syntax-mismatching code; (3) descriptions of adaptation rules for the syntax mismatches; (4) one-shot example sourced from official documentation illustrating the adaptation rule; (5) few-shot examples extracted by `ArkAapter`; and (6) API replacements in the ArkTS built-in module for Node.js built-in module dependencies. We keep components (1), (3), and (6) fixed as the fundamental prompt elements and vary the combinations of components (2), (4), and (5) discovered by `ArkAdapter` to create four prompting methods:
  - **`ArkAapter`*'s Prompting method:*** It includes components (1), (2), (3), (5) and (6).
  - ***Prompting method 1:*** It is the `ArkAapter`'s prompting method without component (2).
  - ***Prompting method 2:*** It is the `ArkAapter`'s prompting method without component (5).
  - ***Prompting method 3:*** It is achieved by replacing component (5) with component (4) from `ArkAapter`'s prompting method.

  By comparing the output results of the four prompting methods, we can analyze the contribution of each varied component to the effectiveness of `ArkAdapter`.
- ***By Priority:*** In our experiments, we compare the effectiveness of adapting syntax-mismatching code with and without our proposed adaptation priority strategy.

Evaluation Metrics. We consider four metrics in the evaluation:

- ***Accuracy exact match (AEM) (%).*** This metric is used to determine the percentage of samples where the inferred adaptations match lexically with the actual adaptations.
- ***Accuracy plausible match (APM) (%).*** This metric measures the percentage of samples where the inferred adaptations match the actual adaptations and successfully pass compilation and test validation. We follow ArkTS and TS/JS standards, manually verifying if the inferred adaptations are similar to the actual adaptations.

- ***Edit distance (ED).*** For the inferred adaptations that are deemed to plausibly match the actual adaptations, We use *ED* to measure the edit operations needed for the inferred adaptations to align with the actual adaptations. A lower edit distance indicates that the inferred adaptations are closer to the actual adaptations [24].
- ***Inference time (IT).*** This is the time in seconds to generate the inferred adaptations for a file.

Results. **`ArkAapter`'s prompting method combined with the *DeepSeek Coder* model outperforms all other baselines.** Table 3 shows the experiment results of `ArkAapter` and baselines across three LLMs and four prompting methods. As observed, 58.89% of code adaptations inferred by `ArkAapter` with *DeepSeek-V2.5-Chat* exactly match the actual adaptations made by OpenHarmony developers. Overall, the *AEM* metric of `ArkAapter` with *DeepSeek-V2.5-Chat* is 6.30% higher than that with *qwen2.5-coder-7b-instruct* and 3.44% higher than that with *gpt-4o-mini*. **80.14% of code adaptations inferred by `ArkAapter` with *DeepSeek-V2.5-Chat* plausibly match the actual adaptations.** On average, *Edit distance* between plausible adaptations and actual adaptations is 6.49. `ArkAapter` with *DeepSeek-V2.5-Chat* outperforms *qwen2.5-coder-7b-instruct* and *gpt-4o-mini* by 10.56% and 4.26% on average *APM* metric, respectively. **Notably, our proposed adaptation priority strategy leads to an average improvement of 9.36% for `ArkAdapter` on the *APM* metric.** The 169 test cases included in benchmark cover 38 adapted ArtTS built-in module API calls. The test verification confirms the correctness of the discovered API migration mappings.

Prompt components (2) and (5) make significant contribution to the *AEM* and *APM* metrics. Comparing the four prompting methods, we can observe that on average, prompt component (2) (contextual code of the syntax-mismatching code) leads to a 4.13% enhancement in the *AEM* metric and a 9.51% enhancement in the *APM* metric. Additionally, compared to *Prompting method 2* (with a zero-shot example), the prompt component (5) (the few-shot examples extracted by `ArkAdapter`) leads to an average improvement of 14.98% in the *AEM* metric and a 22.37% enhancement in the *APM* metric. However, in contrast, when using *Prompting method 3* (with a one-shot example from official documentation), `ArkAdapter` with *DeepSeek-V2.5-Chat* achieves an *APM* metric of 72.24%, which is 7.9% lower than the prompting method that integrated our collected few-shot examples.

On average, `ArkAdapter` with *DeepSeek-V2.5-Chat* requires 6.24 ± 1.76 attempts to resolve compilation failures (*CAttempt*) and 7.35 ± 5.43 attempts to resolve test failures (*TAttempt*), taking 10.81 ± 4.94 seconds to produce results for a TS/JS file. The number of attempts is correlated with the complexity of the adaptation rules. Specifically, single-line, single-hunk, and multi-hunk syntax mismatches require an average of 2.05 ± 0.96, 4.0 ± 2.63, and 5.13 ± 3.10 *CAttempt*, respectively, to achieve the correct code adaptations.

Adaptation Success Rate for Mismatches of Each Syntax Rule. Figure 6(a) illustrates the adaptation success rate for each syntax rule when its mismatch occurs individually within a code block (under the configuration of *DeepSeek-V2.5-Chat*). In our benchmark, the mismatches of 39 syntax rules achieve a 100% adaptation success rate. This success can be attributed to the extensive adaptation knowledge repository of `ArkAdapter` and its effective adaptation priority strategy. For the 994 syntax-mismatching code blocks in the benchmark affected by the remaining 40 syntax rules, our tool achieved an average adaptation success rate of 77.3%. We analyzed 226 failure cases, categorizing the causes into three types:

- ***Lack of Type Inference Clues.*** 117 mismatches of 6 syntax rules causing adaptation failures due to a lack of type inference clues, with an average success rate of 78.3%. ArkTS code prohibits the use of uncertain static types such as *any* and *unknown* found in TS/JS syntax. The `ArkAdapter` utilizes LLM to combine the syntax-mismatching code block with contextual information provided in the prompt to infer the types of uncertain variables in TypeScript/JavaScript code. However,

Table 3. The effectiveness of ArkAdapter in code adaptation along with the baselines

LLM	Prompting Method	By Priorty	AEM (%)	APM (%)	ED	IT (seconds)	CAttempt	TAttempt
ChatGPT gpt-4o-mini	ArkAdapter's Method	yes	55.45%	75.88%	9.07	2.9	6.71	7.59
		no	50.26%	66.05%	11.62	3.96	7.15	8.24
	Method 1	yes	50.97%	67.71%	11.22	2.33	7.01	9.03
		no	48.43%	59.66%	14.04	3.87	7.58	9.12
	Method 2	yes	38.23%	52.70%	15.09	0.86	8.08	9.36
		no	37.75%	47.62%	19.47	1.427	8.04	9.54
	Method 3	yes	50.12%	67.64%	11.44	0.96	7.15	8.17
		no	45.56%	59.19%	14.85	1.76	7.54	8.55
Qwen qwen2.5-coder-7b-instruct	ArkAdapter's Method	yes	52.59%	69.58%	12.24	1.94	7.07	8.03
		no	44.17%	55.70%	16.98	2.63	7.76	8.54
	Method 1	yes	47.34%	61.49%	13.56	1.71	7.53	8.59
		no	42.04%	48.66%	16.76	2.86	8.23	9.65
	Method 2	yes	37.62%	48.38%	18.57	0.76	8.34	9.42
		no	33.78%	38.62%	19.79	1.53	9.54	9.91
	Method 3	yes	48.66%	61.44%	11.92	1.59	7.43	8.56
		no	41.11%	50.94%	16.56	2.21	8.10	9.01
DeepSeek DeepSeek-V2.5-Chat	ArkAdapter's Method	yes	58.89%	80.14%	6.49	10.81	6.24	7.35
		no	55.15%	74.89%	10.34	16.11	6.87	7.81
	Method 1	yes	54.62%	70.58%	10.04	9.00	7.11	8.05
		no	48.34%	60.09%	13.35	16.26	7.46	8.93
	Method 2	yes	41.45%	53.46%	15.61	2.17	7.86	9.04
		no	37.78%	47.23%	18.60	8.79	8.43	9.76
	Method 3	yes	53.56%	72.24%	10.09	3.67	6.93	8.07
		no	45.85%	60.31%	15.14	9.33	7.70	8.43

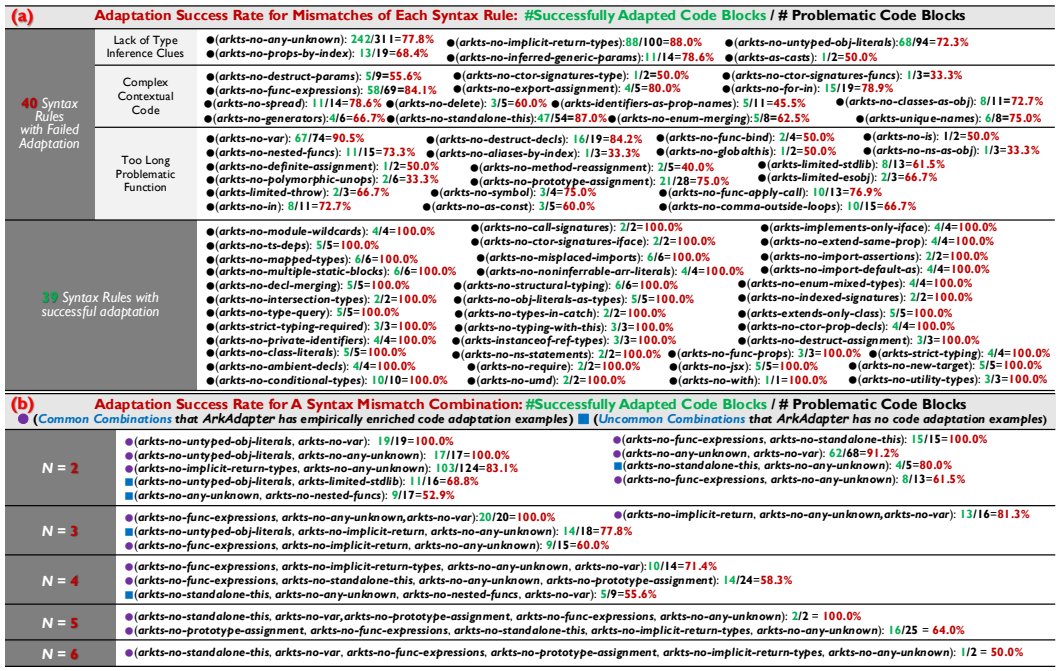


Fig. 6. Adaptation Success Rate for Mismatches of Each Syntax Rule / A Syntax Mismatch Combination

when the clues necessary for type inference are located in annotations or type declaration files instead of being explicitly included in the prompt, it can result in adaptation failures.

- ***Complex Contextual Code.*** 51 mismatches of 14 syntax rules causing adaptation failures due to the complex contextual code structure, with an average success rate of 77.2%. To prevent excessive prompt information from causing the LLM to generate hallucinations, we limited the number of tokens for contextual code in the prompt. ArkAdapter extracts only the code that directly depends on syntax-mismatching code, as well as the code that is directly dependent on it, to serve as contextual information. However, if key information required for adaptation is found in context that is indirectly related to the syntax-mismatching code, this may result in adaptation failures.

- ***Too Long Problematic Function.*** There are 20 syntax rules that affected 73 overly long functions, with 58 adaptations failing, resulting in an average success rate of 74.9%. Too long functions not only increase code complexity but also involve more contextual code. These long syntax-mismatching functions and their corresponding prompts can hinder the LLM's semantic understanding of the code structure and tasks, thus affecting the accuracy of code adaptation. The average number of lines in the failed adaptations of long functions is 336.49 ± 96.53, while the average number of related contextual code lines is 361.89 ± 112.75, leading to a total of 19,238.44 ± 4,207.67 prompt tokens input to the LLM.

Adaptation Success Rate for Combinations of Syntax Mismatches. Figure 6(b) illustrates the adaptation success rate for combinations of syntax mismatches that occur simultaneously in a code block (under the configuration of *DeepSeek-V2.5-Chat*). Our benchmark involves 19 combinations of syntax mismatches. Specifically, **14 out of 19 are common combinations for which `ArkAdapter` has empirically enriched code adaptation examples in our knowledge repository, achieving an average success rate of 82.6%.** In contrast, 5 out of 19 are uncommon combinations, for which our priority adaptation strategy directs `ArkAdapter` to address these mismatches based on the granularity of the affected code, from smallest to largest: *Type System* > *Variable Declaration* > *Object Mechanism* > *Expression Statement* > *Function Declaration* > *Interface Declaration* > *Class Declaration* > *Module System*. **This strategy effectively mitigates interference between syntax mismatches, resulting in an average adaptation success rate of 66.2% for each uncommon combination.** Through case analysis, we identified the primary cause of adaptation failures: *When multiple syntax mismatches occur simultaneously in a code block, if the adaptation of one syntax mismatch depends on the resolution of another and the dependent adaptation cannot be executed, it results in the failure of that combination.* For example, in the benchmark, 15 code blocks are affected by the combination of (***arkts-no-func-expressions***, ***arkts-no-implicit-return-types***, ***arkts-no-any-unknown***). In 6 out of these 15 code blocks, the adaptations for the first two syntax mismatches rely on the adaptation result of the third mismatch. However, the adaptation for ***arkts-no-any-unknown*** fails due to our tool's lack of critical clues for type inference, resulting in an average success rate of 40.0% for this combination.

Conclusion: `ArkAdapter`'s prompting method combined with the *DeepSeek Coder* model outperforms all other baselines. 80.14% code adaptations inferred by `ArkAapter` with *DeepSeek-V2.5-Chat* exactly or plausibly match the actual adaptations made by OpenHarmony developers.

4.5 RQ3: Usefulness of ArkAdapter

Study Methodology. We utilize `ArkAdapter` to port popular TS/JS libraries, and upload the ported ArkTS libraries to the OHPM central repository for validation by official reviewers. We evaluate the usefulness of `ArkAdapter` according to the validation feedback from reviewers. In RQ3, we established the criteria for selecting porting targets from the *npm* repository as follows: (1) The TS/JS library does not depend on any other third-party libraries, or the dependent libraries have already been ported to the OHPM central repository. This helps to prevent compilation errors caused by missing dependencies supported by OpenHarmony. (2) The TS/JS library has a weekly download count exceeding 10,000. Prioritizing the porting of popular libraries contributes to building the OpenHarmony software ecosystem. (3) The TS/JS library is not within the scope of subjects from which we collect code adaptation examples. This helps prevent the issue of overfitting in the evaluation results. Following the criteria, we finally chose 79 subjects.

We upload the successfully ported ArkTS libraries to the OHPM central repository only if they do not cause compiler errors and can pass the associated test cases. To avoid legal risks, we ensure that the post-ported library version uses the same open-source license as the pre-ported library version and state its origin in the project description. To facilitate validation by OHPM central

Table 4. 79 TS/JS Packages ported by ArkAdapter

ID	Project	#mismatches	#APIs	#Files	#Lines	ID	Project	#mismatches	#APIs	#Files	#Lines
1	✔ nearest-periodic...	354/354	113/113	5	22	41	✔ random-item	18/18	0/0	5	23
2	✔ gl-vec3	349/349	46/46	50	909	42	✔ string-natural...	18/18	1/1	5	165
3	✔ skipped-periodic...	291/291	61/61	5	31	43	✔ tinyhttp-for...	17/17	0/0	5	100
4	✔ csstools-color-hel...	255/255	0/0	6	155	44	✔ haversine-distance	17/17	0/0	6	183
5	✔ gl-mat3	246/246	19/19	23	622	45	✔ murmur-128	17/17	0/0	5	58
6	✔ gl-quat	233/233	27/27	31	650	46	✔ yoctocolors	17/17	1/1	6	202
7	✔ ip6	233/233	0/0	6	174	47	✔ middy-http-event...	16/16	0/0	5	318
8	✔ fastest-levenshtein	210/210	0/0	5	103	48	✔ ip-regex	16/16	0/0	5	10
9	✔ gl-vec4	190/190	26/26	30	486	49	✔ mdn-data	16/16	4/4	26	51
10	✔ base64	187/187	5/5	5	146	50	✔ has-flag	15/15	1/1	6	51
11	✔ suncalc	165/165	0/0	5	460	51	✔ is-number	15/15	1/1	5	18
12	✔ punycode	121/121	7/7	5	470	52	✔ leven	15/15	0/0	5	80
13	✔ bresenham	118/118	11/11	5	42	53	✔ css-color-names	14/14	0/0	5	2
14	✔ median-quickselect	99/99	11/11	5	63	54	✔ text-hex	14/14	3/3	5	18
15	✔ frac	69/69	3/3	5	78	55	✔ const-pinf-float64	13/13	1/1	5	1,586
16	✔ fast-blurhash	56/56	9/9	5	144	56	✔ is-primitive	13/13	1/1	5	14
17	✔ fast-levenshtein	51/51	10/10	5	146	57	✔ s3encode	13/13	0/0	5	66
18	✔ matchit	51/51	0/0	5	162	58	✔ ansi-regex	12/12	0/0	5	11
19	✔ math-interval...	51/51	32/32	5	107	59	✔ elegant-spinner	12/12	1/1	5	12
20	✔ brace-expansion	48/48	2/2	5	261	60	✔ known-css-pro...	12/12	1/1	6	4
21	✔ polylabel	48/48	0/0	5	185	61	✔ random-int	12/12	0/0	5	13
22	✔ eslint-plugin...	44/44	4/4	10	43	62	✔ sort-scripts	12/12	4/4	5	43
23	✔ ip-bigint	42/42	0/0	5	66	63	✔ tinyhttp-encode...	11/11	0/0	5	10
24	✔ tinyqueue	34/34	0/0	5	77	64	✔ tinyhttp-url	11/11	4/4	5	19
25	✔ encode-utf8	32/32	4/4	5	53	65	✔ fmix	11/11	0/0	5	59
26	✔ percentile	28/28	1/1	5	124	66	✔ arr-every	10/10	1/1	5	13
27	✔ dargs	26/26	0/0	5	138	67	✔ mathml-tag...	10/10	0/0	5	196
28	✔ hyphenate-style...	25/25	1/1	5	41	68	✔ round-to	10/10	0/0	5	60
29	✔ levenshtein-edit...	25/25	7/7	5	69	69	✔ unicode-match...	10/10	2/2	6	792
30	✔ camelcase-css	23/23	2/2	5	23	70	✔ vendors	10/10	0/0	5	22
31	✔ balanced-match	22/22	0/0	5	89	71	🐞 npmcli/node-gyp	10/19	0/6	4	25
32	✔ lcm	20/20	1/1	5	9	72	🐞 deep-eql	63/138	0/0	4	552
33	✔ yn	20/20	0/0	6	150	73	🐞 direction	12/17	0/5	6	93
34	✔ guess-json-indent	19/19	0/0	5	75	74	🐞 kuler	55/57	1/1	4	152
35	✔ postcss-replace...	19/19	0/0	5	40	75	🐞 postcss-discard...	17/19	0/1	4	95
36	✔ quickselect	19/19	0/0	5	77	76	🐞 postcss-font-var...	20/38	0/2	4	185
37	✔ flush-promises	18/18	6/6	7	18	77	🐞 postcss-media-min...	31/46	0/2	5	164
38	✔ gcd	18/18	1/1	5	7	78	🐞 postcss-opacity-per...	14/17	0/2	4	61
39	✔ hex-rgb	18/18	0/0	5	83	79	🐞 yoctocolors-cjs	30/51	0/2	5	322
40	✔ postcss-page...	18/18	2/2	5	147						

✔: The ported ArkTS library has been approved by OHPM reviewers; 🐞: Compilation errors or test failure;
#mismatches: (#Adapted Syntax Rules / #Total Mismatched Syntax Rules); ***#Files***: #Adapted Files; ***#Lines***: #Total Adapted Code Lines;
#APIs: (#Migrated Node.js built-in module APIs / #Total invoked Node.js built-in module APIs);

repository reviewers, all ArkTS libraries adhere to the community's porting rules and come with log files of the compilation process and test case execution.

Results. Table 4 summarizes the experiment results of RQ3. Using ArkAdapter, we analyzed 79 TypeScript and JavaScript libraries consisting of 48,535 lines of code. On average, the source code of each TS/JS library mismatched 56.18 ArkTS syntax rules and made calls to 5.80 Node.js built-in module APIs that need to be migrated. The tool adapted a total of 10,944 lines of code from 540 source files, successfully porting 70 TS/JS libraries to the OpenHarmony platform. These 70 ported libraries passed both the Arkcompiler validation and test case verification, and were all approved by OHPM central repository reviewers.

We identified three main failure patterns in the remaining 9 libraries: (1) Two libraries (*yoctocolors*[23] and *direction*[8]) lacked ArkTS API equivalents for Node.js dependencies; (2) Three (*kuler*[13], *postcss-media*[20], and *cssnano*[6]) failed due to complex context structures; (3) Four (*postcss-font-variant*[19], *postcss-opacity*[21], *deep-eql*[7], and *node-gyp*[15]) exceeded LLM comprehension limits (averaging 167.32 lines with 15.25 syntax mismatches).

Feedback from OHPM central repository Reviewers. We will discuss representative cases of reviewers' feedback that highlight the the usefulness of ArkAdapter in two key aspects:

- ***Porting popular libraries is beneficial for the growth of the OpenHarmony software ecosystem.*** These 70 ported libraries, with an average weekly download count of 16,844k on the *npm* repository, received 21 "*Likes*" from OpenHarmony developers within the first week after being uploaded to the OHPM central repository, accumulating 1,650 downloads. During the review process of libraries *ip6* [11], *punycode* [22], and *encode-utf8* [9], developers provided

feedback stating: "*The software libraries offer valuable functionality support for HarmonyOS app development in* network communication*,* character conversion*, and* format encoding *aspects!*"

- **`ArkAdapter` *adheres to the porting rules defined by the OHPM central repository for the ported software libraries.*** All 70 ArkTS libraries we ported achieved the maximum OHPM score (50/50), significantly outperforming the community average (33.35±10.83, n=839). OHPM reviewers verified each submission, including compilation logs and test results.

Conclusion: *We utilized* `ArkAdapter` *to port 79 TS/JS libraries, out of which 70 ported ArkTS libraries (88.6%) that passed the compilation and test validation were approved by the OHPM reviewers. The feedback indicates the usefulness of our porting technique.*

5 Discussions

5.1 `ArkAdapter`'s Generalizability Beyond the Evolution of ArkTS Syntax Features

As the OpenHarmony community matures, the features of the ArkTS language and its syntax adaptation rules may evolve. In fact, `ArkAdapter` exhibits good generalizability beyond the evolution of ArkTS syntax features for the following two reasons:

- `ArkAdapter` leverages *ArkCompiler* to identify syntax-mismatching code, and *ArkCompiler* will inevitably evolve with the features of the ArkTS language. As a result, the tool can accurately detect mismatches with the updated ArkTS syntax.
- `ArkAdapter` guides the LLM in achieving code adaptation by parsing 79 adaptation rules from the official documentation and building a knowledge repository that includes adaptation examples for complex and diverse scenarios. Our tool can automatically update adaptation rules by analyzing the documentation. As the OHPM community grows, we can use automated techniques to find real adaptation code examples for different scenarios, gradually enhancing the knowledge repository. Evaluation results indicate that for complex combinations of syntax mismatches without code examples, our priority strategy can still effectively address most issues.

5.2 Threats to Validity

Data leakage. "*Data leakage*" refers to evaluating a dataset that was included in the LLMs' training data, which can cause overfitting and biased results. However, ArkTS is a new programming language with a growing ecosystem. Our data shows that only 19 ArkTS libraries in the OHPM repository are clearly linked to their original TS/JS versions before porting. While the benchmarks in RQ2 may be part of the training sets for popular LLMs covering millions of libraries in established languages like Java and C/C++, the impact of data leakage on the RQ2 evaluation is minimal.

Reproducibility. The randomness in the content generation by LLMs could potentially impact the reproducibility of `ArkAdapter`. Following the previous approach [29], we conducted all experiments ten times and computed the average results to mitigate the influence of LLMs' randomness. On the other hand, *DeepSeek Coder*, which performed the best in the experiments, is an open-source LLM. The version we adopted will not be deprecated, ensuring the reproducibility of the experiments.

6 Related Work

6.1 Code Translation with LLMs

Existing research on LLM-based code translation falls into two main categories: inference-based and training-based. Inference-based studies focus on improving an LLM's performance in code translation without retraining or fine-tuning. Yang et al. [41] conducted an empirical study on five LLMs with Python, Java, and C++, finding that these LLMs outperform traditional transpilers. They also noted that performance improves with testing on translated code and providing feedback. Pan et al. [32] examined bugs introduced by LLMs in code translation for C, C++, Go, Java, and Python, categorizing them into 15 types and suggesting that LLM-based and non-LLM-based methods can

complement each other. Hong et al. [25] studied LLM performance in translating C code to Rust, finding that direct instructions often lead to type errors. They proposed a prompting strategy to encourage the use of Rust idioms, which reduces type errors. Jiao et al. [27] introduced a benchmark for code translation among C++, Java, C#, JavaScript, and Python.

Training-focused research [33–35, 43] explores the performance and strategies involved in retraining or fine-tuning a LLM. However, these studies diverge from the focus of this paper, as `ArkAdapter` follows an inference-based approach. To the best of our knowledge, no existing work has addressed the task of translating code from TS/JS to ArkTS. This task presents unique challenges not encountered in translating between more common languages, including a scarcity of training data, insufficient documentation, and distinctive syntax rules as outlined in Section 1. Moreover, while previous works such as [25, 27, 32, 41] concentrate on function-level or code-snippet-level translation, our research is centered on project-level translation, introducing more complexity.

6.2 Bug Repair with LLMs

`AlphaRepair`[39] is the first method using LLM for program repair in a cloze-style approach. It replaces faulty lines with masked tokens and asks the LLM to generate the correct tokens. Xia et al.[38] studied four LLMs and found that fine-tuning them with project-specific information improves performance. Huang et al.[26] also examined fine-tuning LLMs for different types of bugs and offered advice on creating effective fine-tuning datasets, such as using tree or graph structures for code. Wei et al. [36] combine code completion engine and LLM to enhance LLM's capability in generating syntactically correct patch. Xia et al. [40] propose `ChatRepair`, a pipeline that iteratively prompts ChatGPT to generate patches. In each iteration, `ChatRepair` first prompts ChatGPT to generate a patch. Then, it validates the generated patch through testing. Upon test failure, `ChatRepair` provides the failure information to ChatGPT to further refine the patch.

`ArkAdapter` uses an iterative prompting strategy to fix syntax mismatches, similar to bug repair technique `ChatRepair` [40]. However, it differs in three main ways: (1) Unlike traditional methods that assume prior knowledge of bug locations, `ArkAdapter` identifies syntax mismatches in the adapted code using *ArkCompiler* and program analysis. (2) While conventional techniques often focus on single-line bugs, `ArkAdapter` can address multi-hunk issues. (3) Instead of targeting function or file levels, `ArkAdapter` is designed for project-level porting, fixing multiple compilation errors by prioritizing syntax mismatches based on code dependencies and mismatch granularity.

7 Conclusion and Future Work

In conclusion, `ArkAdapter` presents an automated solution for porting TS/JS libraries to OpenHarmony. It achieves this by establishing an adaptation knowledge repository for ArkTS syntax comprehension, expanding real-world code adaptation instances to enhance LLMs' capabilities through few-shot learning, and prioritizing adaptation to prevent code block interference, thus reducing the risk of introducing new syntax mismatches. It achieves high precision and recall rates, aiding in the location of syntax-mismatching code effectively. In the future, we will further improve the `ArkAdapter`'s generalizability beyond the evolution of ArkTS syntax features.

8 Data Availability

We provided a reproduction package, including the above datasets, an available tool, and experiment raw data, on the website (***http://arkadapter-library.com/***) for facilitating future research.

Acknowledgement

This work was supported by the National Key Research and Development Program of China (Grant No. 2024YFF0908000) and the China Postdoctoral Science Foundation (Grant No. 2024M750375).

References

[1] 2024. Adaptation Rules from TypeScript/JavaScript to ArkTS Language:. https://gitee.com/openharmony/docs/blob/master/zh-cn/application-dev/quick-start/typescript-to-arkts-migration-guide.md. Accessed: 2024-09-01.

[2] 2024. ArkAnalyzer tool:. https://portrait.gitee.com/openharmony-sig/arkanalyzer?skip_mobile=true. Accessed: 2024-09-01.

[3] 2024. ArkCompiler:. https://developer.huawei.com/consumer/cn/arkcompiler/. Accessed: 2024-10-15.

[4] 2024. chardet:. https://www.npmjs.com/package/chardet. Accessed: 2024-10-15.

[5] 2024. Code Generation Leaderboard:. https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard. Accessed: 2024-10-15.

[6] 2024. cssnano:. https://www.npmjs.com/package/cssnano, note = Accessed: 2024-10-15.

[7] 2024. deep-eql:. https://www.npmjs.com/package/deep-eql. Accessed: 2024-10-15.

[8] 2024. direction:. https://www.npmjs.com/package/direction. Accessed: 2024-10-15.

[9] 2024. encode-utf8:. https://www.npmjs.com/package/encode-utf8. Accessed: 2024-10-15.

[10] 2024. flexsearch:. https://www.npmjs.com/package/flexsearch. Accessed: 2024-10-15.

[11] 2024. ip6:. https://www.npmjs.com/package/ip6. Accessed: 2024-10-15.

[12] 2024. js-e2e tool:. https://gitee.com/chenwenjiehw/js-e2e/tree/master/js-compatibiitiy. Accessed: 2024-09-01.

[13] 2024. kuler:. https://www.npmjs.com/package/kuler. Accessed: 2024-10-15.

[14] 2024. Library Porting Specifications:. https://gitee.com/openharmony-tpc/docs/blob/master/contribute/adapter-guide. Accessed: 2024-09-01.

[15] 2024. node-gyp:. https://www.npmjs.com/package/node-gyp. Accessed: 2024-10-15.

[16] 2024. OHPM:. https://ohpm.openharmony.cn/#/cn/home. Accessed: 2024-10-15.

[17] 2024. OpenAtom Foundation:. https://en.wikipedia.org/wiki/OpenAtom_Foundation. Accessed: 2024-10-15.

[18] 2024. pinyin4js:. https://www.npmjs.com/package/pinyin4js. Accessed: 2024-10-15.

[19] 2024. postcss-font-variant:. https://www.npmjs.com/package/postcss-font-variant. Accessed: 2024-10-15.

[20] 2024. postcss-media-minmax:. https://www.npmjs.com/package/postcss-media-minmax. Accessed: 2024-10-15.

[21] 2024. postcss-opacity-percentage:. https://www.npmjs.com/package/postcss-opacity-percentage. Accessed: 2024-10-15.

[22] 2024. punycode:. https://www.npmjs.com/package/punycode. Accessed: 2024-10-15.

[23] 2024. yoctocolors:. https://www.npmjs.com/package/yoctocolors. Accessed: 2024-10-15.

[24] Yangruibo Ding, Baishakhi Ray, Premkumar Devanbu, and Vincent J Hellendoorn. 2020. Patching as translation: the data and the metaphor. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 275–286.

[25] Jaemin Hong and Sukyoung Ryu. 2025. Type-migrating C-to-Rust translation using a large language model. *Empirical Software Engineering* 30, 1 (2025), 3.

[26] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An empirical study on fine-tuning large language models of code for automated program repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1162–1174.

[27] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541.

[28] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé F Bissyandé, et al. 2023. Software engineering for openharmony: A research roadmap. *arXiv preprint arXiv:2311.01311* (2023).

[29] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 14–26.

[30] Mingwei Liu, Yanjun Yang, Yiling Lou, Xin Peng, Zhong Zhou, Xueying Du, and Tianyong Yang. 2023. Recommending analogical APIS via knowledge graph embedding. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1496–1508.

[31] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.

[32] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[33] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *Advances in neural information processing systems* 33 (2020), 20601–20611.

[34] Baptiste Roziere, Jie M Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. 2021. Leveraging automated unit tests for unsupervised code translation. *arXiv preprint arXiv:2110.06773* (2021).

[35] Marc Szafraniec, Baptiste Roziere, Hugh Leather, Francois Charton, Patrick Labatut, and Gabriel Synnaeve. 2022. Code translation with compiler representations. *arXiv preprint arXiv:2207.03578* (2022).
[36] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.
[37] Rongxin Wu, Hongyu Zhang, Sunghun Kim, and Shing-Chi Cheung. 2011. Relink: recovering links between bugs and changes. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 15–25.
[38] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). *IEEE, Melbourne, Australia* (2023), 1482–1494.
[39] Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *2022 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.
[40] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.
[41] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and unleashing the power of large language models in automated code translation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1585–1608.
[42] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.
[43] Ming Zhu, Karthik Suresh, and Chandan K Reddy. 2022. Multilingual code snippets training for program translation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 11783–11790.

Received 2025-02-22; accepted 2025-03-31